NB-IoT RACH — MATLAB/Octave Implementation Plan

1. Recommended direction

Build an Octave-first, MATLAB-compatible discrete-event simulator with tabular Q-learning as the main controller. Use plain `.m` files and basic array operations. Do not require the Reinforcement Learning, Deep Learning, Communications, or Statistics toolboxes for the main experiments.

In simple terms: first build a simulator that never loses track of a request, then compare several controllers inside that same simulator.

Decisions

Question	Recommendation	Reason
MATLAB or GNU Octave?	GNU Octave as the open reference runtime; keep the core compatible with MATLAB	Others can reproduce the main results without a paid licence. MATLAB can be used for development and an additional compatibility check if available.
Main learning algorithm?	Standard tabular Q-learning, with a decreasing learning rate, feasible-action masking, and frozen-policy testing	Matches the supplied methodology, is easy to inspect, and gives a clear low-complexity starting point.
Start with DQN, PPO, or multi-agent RL?	No	These add complexity before the simulator and evaluation have been validated. They are not automatically better for this compact decision problem.
Deep-RL comparison?	Optional second-stage experiment: one genuinely trained DQN	Include only if there is enough time to implement, tune, and test it fairly. Otherwise make no claim of outperforming deep RL.

Question	Recommendation	Reason
Reuse the old monolithic script?	Keep it as an archived reference; build the new environment in separate modules	The new request accounting, timing, metrics, and baseline rules are substantial changes. Old reported numbers are not acceptance targets.
Initial claim?	"An NB-IoT-inspired, system-level evaluation under stated assumptions"	Selected values can match a standard without the complete simulator, admission mechanism, or update schedule being standards-compliant.

"Best" means best justified starting design for this project—not an algorithm or parameter set already proved optimal. Final choices must be selected using training/validation data, never the final test results.

Inputs used

- The supplied `FINAL_NB_IoT_RACH_Methodology.pdf`, including its request model, reward, baselines, metrics, and validation gates.
- The rejected manuscript and both DELCON reviewers' comments.
- The earlier problem/literature/research-gap review and the inspected repository papers and MATLAB script.
- Additional checks against Release 15 MAC and RRC specifications and the NB-IoT primer, cited below.

Excluded: the novelty-claims DOCX, as requested. No simulator has been implemented or training run for this plan. Neither MATLAB nor Octave was found on the current computer's command path; runtime installation or another execution machine is needed before coding tests can be run.

2. Define exactly what the simulator will measure

Use three development levels. Do not mix their results in a table without labelling them.

Level A — Correct contention model

Model admissions, singleton preamble outcomes, failed attempts, and retry timers. This is for unit tests, analytical checks, and debugging.

Endpoint: preamble success only. Do not call it full network access or application delivery.

Level B — Main study: simplified random-access completion

Add explicit stages for preamble transmission, Random Access Response, Msg3, and contention resolution. Include queues, delays, timeouts, and finite service capacity where relevant. A request becomes COMPLETED only at the declared final access stage.

Endpoint: completion of the represented random-access procedure. This is the recommended primary experimental level.

These stages follow the four-step contention-based procedure described in the NB-IoT primer and the MAC specification. Their detailed timing and capacities still need to be mapped or clearly labelled as study assumptions.[1][2]

Level C — Optional application-message delivery

Add the actual data-service queue, packet size, scheduling, retransmissions, and delivery endpoint. Only this level can support application-message delivery claims. Do not substitute a constant unexplained delay for these stages and call the result a utility SLA.

Important: Level B still needs uplink resource accounting for Msg3. A general NPRACH-versus-user-data throughput claim requires Level C or an explicitly modelled competing-data workload.

3. Starting system parameters

These are **proposed defaults and search ranges**, not final optimal settings. Mark each configuration field as `verified_value`, `study_assumption`, or `to_be_mapped`.

Parameter	Starting setting	Sensitivity / validation
Cell and carrier	One cell; one 180 kHz uplink carrier	No hidden parallel carriers. Add multi-carrier operation only as a separate extension.

Parameter	Starting setting	Sensitivity / validation
Standard reference	Release 15 FDD, legacy NPRACH configuration family	Do not mix FDD, TDD, and format-2 parameter lists.
Coverage	One good-coverage group initially	Add separate repetition/coverage scenarios later; do not assume the initial case validates deep coverage.
Population	50,000 meters for the reference scenario	Debug at 100 and 1,000; evaluate 10,000, 50,000, and 100,000.
Reporting period	900 s	Study assumption, not an IEC-mandated interval. Test 300 and 1,800 s separately.
Reporting phase	Uniform initial phase across the reporting period; zero extra jitter initially	Add synchronised phases and declared jitter distributions as stress tests.
Reference event	Starts at 30 s; 5% of meters affected; requests spread over 10 s	At 50,000 meters this creates 2,500 event requests, in addition to periodic reporting.
Event timing	Beta(3,4) within the declared event window	Also test uniform and near-simultaneous events. Generate timestamps, not rounded bin totals.
Periodic deadline	60 s after generation	Study choice; test 30 and 120 s in separate scenarios.
Event deadline	30 s after generation	Study choice pending an actual utility requirement; test 10 and 60 s.
Arrival window	Generate requests during [0,120 s)	For the default deadlines, continue observation through 180 s so every generated request’s deadline matures.
Long-run reporting test	At least two full reporting periods, with a stated warm-up	Required separately; a 120 s event experiment does not establish long-term periodic-load stability.
NPRACH period	80 ms	Test 40 and 160 ms as separate fixed scenario configurations. Do not retain the old 50 ms value as a verified FDD NPRACH periodicity.

Parameter	Starting setting	Sensitivity / validation
NPRACH start offset	Candidate 8 ms within the period	Validate the complete timing configuration, including repetitions and non-overlap.
Configured subcarriers	Candidate choices 12, 24, 36, 48	Do not automatically equate configured subcarriers with usable contention choices. Record the CBRA start-subcarrier mapping.
Initial preamble repetition	1 for an ideal good-coverage case	Verify the exact selected configuration; then test legal repetition settings with corresponding time/resource costs.
Retry limit	Initially 10 total preamble transmissions per request	Explicit study rule. Do not call it a fully mapped 3GPP retry procedure until per-CE and total counters are implemented. Test 5 and 20 as abstract sensitivity settings.
Per-meter concurrency	One active access procedure per meter; later messages wait in FIFO order	Keep separate message IDs. No silent coalescing or simultaneous procedures from one simulated UE.
Queue capacity	No artificial cap in the reference experiment; preallocate storage for the known trace	An optional finite-capacity scenario must record every overflow as a drop.
Deadline expiry behaviour	Record a miss but continue serving the request	This permits measuring late completion and drain time. Expiry-as-drop is a separate policy/scenario, not a hidden default.
Channel assumption	Ideal reception for the initial Level A tests	Later add declared failure/detection models. Do not present arbitrary failure probabilities as measured NB-IoT propagation.

The primer supports the 180 kHz uplink abstraction. TS 36.331 lists 40/80/160 ms among FDD NPRACH periodicities, 8 ms among start offsets, and 12/24/36/48 among configured subcarrier values. It separately defines contention-based starting-subcarrier settings. These facts support candidate values, not the legality of every combination or dynamic reconfiguration schedule.[1][3]

Parameters that must not be invented

Before Level B comparative runs, complete a protocol configuration sheet for:

- Preamble format, repetition duration, hopping/resource reservation, and usable CBRA choices.
- RAR response window, monitoring periods, grant capacity, and timeouts.
- Msg3 resource allocation, service capacity, and failures.
- Contention-resolution timing, capacity, and retry rules.
- Exact mechanisms and effective times for admission, BI, and NPRACH changes.

Use the selected release's MAC, RRC, physical-channel, and physical-procedure specifications and cross-check against the validated-model literature. If an item cannot be mapped, use a clearly named abstract assumption and test sensitivity; do not disguise a guessed value as a standard parameter. Martiradonna et al.'s validated NB-IoT modelling work is an important cross-check.[4]

4. Choose the action space carefully

Initial abstract controller grid

Use admission probability `pA`, not barring probability:

```
pA_values = [0.001 0.005 0.01 0.025 0.05 0.10 0.25 0.50 1.0
0];
backoff_ms = [0 256 1024 4096];
nprach_config_ids = [1 2 3 4];
```

The four candidate NPRACH configuration IDs initially represent the separately validated 12/24/36/48-subcarrier configurations, with all their other fields stored in the registry. The candidate Cartesian product contains **9 × 4 × 4 = 144 actions before feasibility filtering**.

- The small admission values allow strong overload regulation; `pA=1` permits unrestricted attempts.

- Keep `pA=0` only as a diagnostic “bar everything” test, not a default deployable action.
- Use random backoff drawn uniformly from 0 to the selected value, then schedule the next eligible opportunity. Do not redraw an existing request’s timer when a later controller action changes the setting.
- BI values 0, 256, 1024, and 4096 ms are entries in the NB-IoT backoff table. BI is delivered through the relevant MAC response mechanism; it is not a universal per-slot SIB setting.[2]

Admission caveat: the Release 15 NB-IoT access-barring procedure has its own MIB/SIB14-NB, access-class/category, and bitmap rules. A freely chosen Bernoulli admission probability must remain an abstract admission controller until an actual signalling and UE-behaviour mapping is demonstrated. Do not simply copy ordinary LTE ACB factor semantics into NB-IoT.[3]

Action registry fields

`action_id, pA, backoff_ms, nprach_config_id, usable_choices, period, start_offset, repetitions, resource_map_id, effective_time_rule, minimum_hold_time, standards_status, source_clause`.

At every decision, filter actions for feasibility. If an NPRACH change cannot yet take effect, hold it fixed and allow only the remaining legal controls to change.

Control timing

For the **abstract reference** experiment, decide every 640 ms—eight 80 ms NPRACH periods. Schedule the chosen configuration to take effect at the next permitted NPRACH boundary, not retrospectively. Retain the previously active settings until that point.

Test additional delays of 0, 640 ms, and 2.56 s, and hold times of 640 ms and 2.56 s, as labelled modelling sensitivities. Zero delay is an idealised upper-reference case. These numbers are **not asserted SI-update schedules**. The standards-mapped mode must use the actual permitted update mechanism and timing; it may require much slower changes or a reduced action set.

Log selected action, active action, pending changes, and effective timestamps separately. For delayed-control experiments, include known action context in a richer-state ablation and report the resulting memory increase.

5. Capacity and resource checks before learning

Simple check for the initial contention abstraction

If the model has 48 usable choices every 80 ms, then 30 seconds contains 375 opportunities.

- Collision-free one-step upper bound: `375 × 48 = 18,000` successes.
- With 48 independent uniform attempts over 48 choices, expected singleton successes per opportunity are `48 × (47/48)^47 ≈ 17.84`.
- At continuously matched load this is about **6,692 preamble successes over 30 seconds**, before later access-stage limits.
- The 50,000-meter, 900 s reporting case contributes about **55.6 new periodic requests/s** on average.

These calculations explain why the reference event begins at a modest 5% affected fraction, and why a simultaneous 50,000-request event cannot be promised full service within 30 seconds in this abstraction. They are not universal NB-IoT capacity results.

Use 1%, 5%, 20%, and 100% event participation to explore different loads. Classify a scenario as feasible, near-capacity, or overloaded **after** checking its actual resource and protocol capacity. Do not label a fraction feasible merely because it sounds small.

Resource accounting

Use a time–frequency reservation calendar. A convenient internal accounting unit is `3.75 kHz × 1 ms`, with exact event timestamps retained separately.

1. Represent the actual reserved NPRACH region, including hopping and repetitions.
2. Convert Msg3 and any user-data allocations to the same unit.
3. Reject overlapping or over-budget allocations.
4. Account for downlink response/confirmation capacity separately; it is not free simply because the objective concerns uplink access.

5. Count reserved resources consistently across policies—not just the resources used by successful devices.

Do not subtract “number of preambles” directly from “data packets.” Different units must not be mixed. Do not impose a general linear NPRACH-to-NPUSCH capacity relation without the configuration calendar that supports it.

6. Simulator architecture and files

Use functions and structs rather than toolbox-specific environments or a large script containing every operation.

```
nbiot_rach/
  run_smoke_tests.m
  run_training.m
  run_evaluation.m
  make_paper_outputs.m
  config/
    default_config.m
    scenario_registry.m
    action_registry.m
    baseline_registry.m
    standards_map.csv
  traffic/
    generate_trace.m
    beta34_inverse.m
    save_trace.m
  environment/
    reset_env.m
    advance_to_observation.m
    schedule_action.m
    simulate_until.m
    process_event_batch.m
    update_request_status.m
    resource_calendar.m
    observe_controller.m
    encode_state.m
    compute_reward.m
    assert_invariants.m
  agents/
    fixed_policy.m
    adaptive_policy.m
    q_select_action.m
    q_update.m
    freeze_policy.m
  evaluation/
    request_metrics.m
```

```
    aggregate_runs.m
    paired_intervals.m
    benchmark_policy.m
  tests/
    test_conservation.m
    test_deadline_boundary.m
    test_contention.m
    test_backoff.m
    test_action_delay.m
    test_reward.m
    test_q_transition.m
    test_reproducibility.m
  traces/
  results/
  figures/
  docs/
    model_assumptions.md
    methodology_to_code.csv
    reviewer_resolution.csv
```

These are proposed filenames, not files already implemented.

Portability rules

- Prefer numeric arrays, logical masks, `struct`, `accumarray`, `sort`, `rand`, and explicit helper functions.
- Use one named function per file; avoid relying on recent MATLAB-only features.
- Store portable numeric checkpoints in an agreed MAT-file format, with CSV summaries and plain-text configuration metadata.
- Provide a small local percentile function rather than requiring a statistics package.
- For Beta(3,4), the CDF is `20u^3 - 45u^4 + 36u^5 - 10u^6`. Invert it using a tested bisection helper and map `t = onset + duration*u`; this avoids a mandatory `betarnd` dependency.
- Use integer time ticks, for example microseconds stored as `int64`, to avoid floating-point deadline comparisons. Record rounding rules for physical durations.

- Verify both runtimes with deterministic test fixtures. Do not promise bit-identical pseudorandom streams between MATLAB and Octave without testing; share saved traces/draws when exact replay is required.

7. Request ledger and event engine

Request fields

`request_id, meter_id, arrival_time, deadline_time, traffic_class, attempts, status, stage, next_eligible_time, completion_time, drop_reason, miss_recorded, drop_recorded`.

Store these as preallocated parallel arrays rather than one large growing struct per request. Use separate per-meter procedure/busy state.

Status rules

- WAITING/BARRED: retained until eligible; a rejected admission draw never deletes a request.
- BACKOFF: retained with one sampled release timestamp.
- ACCESS-IN-PROGRESS: retained through the declared protocol stages.
- COMPLETED: terminal, with an actual completion timestamp.
- DROPPED: terminal, with a reason such as retry limit or explicit overflow.

A missed deadline is a separate recorded event; it is not automatically a new terminal status. Once completed or dropped, a request must never re-enter a queue.

Required invariants

After every event batch:

`generated = completed + dropped + pending`.

Also assert unique IDs, one status per request, no duplicate completion, no terminal-to-pending transition, no overlapping per-meter procedure, and no resource-calendar violation. An assertion failure invalidates the run; save the failing configuration and replay trace rather than quietly continuing.

Deadline-boundary clarification

The methodology says to process matured deadlines before executing the next interval, but also says completion exactly at the deadline is on time. Implement a precise timestamp rule:

1. At time `t`, finish all already scheduled completion events timestamped `t`.
2. Finalise deadline events timestamped `t` only after those completions are known.
3. Form the next decision observation after the declared same-time event batch is complete.
4. Do not allow a future event later than `t` to rescue a request that missed at `t`.

This is a necessary clarification of the methodology, not a change to its definition of on-time service. Test completions at `d−1 tick`, `d`, and `d+1 tick`.

8. Observations and state design

Separate three interfaces: simulator truth, controller observations, and reward/evaluation records. Offline training may use the full ledger to compute the true reward; this does not make hidden ledger information a deployable state input.

Compact starting state: 150 bins

Use three estimated features, with the final definitions frozen before final testing:

Feature	Candidate definition	Initial bin edges
Congestion proxy, 5 bins	EWMA of unsuccessful detected access outcomes divided by detected occupied choices; explicitly a receiver-observation proxy, not the exact number of colliding devices	`[0 .2 .4 .6 .8 1]`
Eligible-load estimate / 48, 5 bins	Estimate contention from observed idle choices and account for the admission probability active during that measurement window	`[0 1 5 20 100 Inf]`
Load trend, 6 bins	Current eligible-load estimate divided by the previous estimate, with an explicit empty-history rule	`[0 .5 .8 1 1.25 2 Inf]`

Start EWMA smoothing at 0.2; validate 0.1 and 0.4. Include last-edge equality, zero denominators, and cold-start tests.

For an idealised idle-count observation, one candidate attempted-load estimate is `log(I/n)/log(1-1/n)` for `0 < I < n`. The admission correction uses the **effective** admission probability, not a future selected action. Smooth or combine opportunity-level estimates; do not invert a window containing different active configurations as if it had one configuration.

If `I=0`, the estimate is saturated—not an exact finite load measurement. Mark saturation, apply a documented cap/prior based on the known population, and test its sensitivity. If `I=n`, use the empty-observation rule. Neither case permits reading true backlog secretly.

Observation limitation: idle/occupied classification and successful outcomes require a stated detector/measurement model. If the receiver model cannot distinguish a collision, use only the outcomes actually available and revise the proxy. Perfect event classification is an ideal-observation assumption, not a proven eNB capability.

Required state comparisons

1. Compact estimated state above.
2. Richer state using permitted history and known active/pending action context.
3. Oracle state containing true pending load and request-age information, clearly labelled as unavailable ground truth.

A finite table does not prove the state is Markov. Describe the compact controller as operating on a quantised observation of a partially observed environment unless stronger assumptions are established.

With 150 states and 144 unfiltered actions, a float32 table uses **86,400 bytes, about 84.4 KiB**, excluding counters and other controller state. A `uint16` frozen action map for 150 states needs 300 bytes for its entries alone, but still requires feature extraction, bin definitions, feasibility handling, and any timing context. Recalculate all sizes after filtering and state changes.

9. Reward and learning settings

Starting reward

Follow the supplied methodology:

`r = C_on/K − M_new/K − X_new/K − 0.01*B_norm − 0.01*R_norm`.

These are initial weights for validation, not proven optimal weights.

- `C_on`: newly completed on-time requests in the interval.
- `M_new`: newly recorded deadline misses, excluding requests already dropped before that deadline event.
- `X_new`: newly terminally dropped requests.
- `K`: fixed at 384 for the reference study—48 reference choices × eight reference opportunities per 640 ms decision window. Keep this scale unchanged when an action chooses fewer resources, and disclose its use if scenario timing changes.
- `B_norm`: true pending request count divided by `N_meters`, without clipping. It may exceed 1 because meters may generate more than one request. This is an offline reward signal, not automatically a controller observation.
- `R_norm`: resources reserved during the interval divided by the declared interval resource budget. Define the treatment of downlink costs separately.

Use event flags so a deadline miss or drop is recorded once. A late request that later drops may incur both one miss cost and one terminal-drop cost; these are distinct costs. State that rule explicitly. Do not repeatedly punish the same miss each interval.

Reward ablations and tuning

Compare:

- Throughput-only: total newly completed requests / K, including late completions.
- Deadline-aware, without resource cost.
- Full reward above.

Tune `w_miss` in `{1,2}`, `w_drop` in `{1,2}`, `w_q` in `{0,0.01,0.05}`, and `w_r` in `{0,0.01,0.05}` using a prespecified validation budget, not a huge uncontrolled search. Fix `w_on=1`.

Check for prolonged barring, starvation of either traffic class, and deadline-expiry gaming.

Learning defaults

Item	Proposed default	Check
Initial Q values	Zero, float32	Random tie-breaking among equal feasible actions during training.
Learning rate	`alpha = (1 + visit_count(s,a))^(-0.6)` after incrementing the visit count	Decreasing per state–action pair; not a convergence guarantee under partial observation.
Discount	0.99 per 640 ms decision	Effective discount horizon is about 64 s; validate against 0.995 and 1.0 in a finite-horizon setting.
Exploration	Linear decrease from 1.0 to 0.05 over the first 80% of training, then hold	Frozen evaluation always uses 0.
Pilot training	100 episodes, then profile and inspect coverage	Debug budget, not a final reported performance claim.
Candidate main budget	Start at 2,000 episodes; permit up to 5,000 under a preregistered validation stopping rule	Select from validation performance and state–action coverage, not attractive-looking reward curves.
Validation checkpoints	Every 100 episodes	Fixed validation traces; choose the checkpoint before opening test results.
Final independent training runs	At least 5 seeds; aim for 10 after profiling	Report uncertainty and failures, not only the best run.

Use the actual next observation. The environment should return `(s_next, reward, terminal, info)` after advancing through the entire interval and preparing the next decision boundary. That same `s_next` is reused at the next decision; do not generate arrivals or deadline events a second time.

For a true terminal transition, the target is `reward` only. Frozen evaluation must not update Q. If finite-horizon end effects matter, test a time-to-end feature or a

continuing-training design; do not assume a stationary 150-state representation fully captures the horizon.

Why not immediately choose a more complex algorithm?

Jiang et al. already compare tabular and neural approaches and investigate larger multi-parameter NB-IoT configurations. That makes tabular Q-learning a justified experimental candidate, not an inherently superior algorithm. If the richer state causes poor coverage, run a genuine DQN comparison or reduce the research scope rather than hiding the state-growth problem.[5]

10. One baseline registry for everything

ID	Method	Fair-comparison rule
B0	Tuned fixed configuration	Select its fixed tuple on training/validation traces from the same action registry. Do not call one arbitrary poor tuple “3GPP performance.”
B1	Load-aware adaptive method	Prefer a source-faithful D-ACB-style implementation. If the NB-IoT admission/timing adaptation changes it, label it an adapted heuristic.
B2	ACB/admission-only tabular Q	Fix backoff and NPRACH to validation-tuned values.
B3	Admission + backoff tabular Q	Fix NPRACH to a validation-tuned configuration.
B4	Admission + NPRACH tabular Q	Fix backoff to a validation-tuned value.
B5	Full joint tabular Q	Same observations, reward family, resource accounting, and test traces as B2–B4.
B6, optional	Actual DQN	Implement, train, tune, checkpoint, and freeze it. No proxy substitution.

D-ACB provides a useful analytical/adaptive source. The simple idea `pA ≈ min(1,n/estimated_eligible)` may be used as an explicitly named occupancy-matching heuristic, but this formula alone is not a faithful reproduction of the complete D-ACB algorithm.[6]

Do not include LE-URC by name unless its actual method is implemented. The previous manuscript's suggested reference-number fix is insufficient; verify the source. Do not add DDQN/PPO merely to make the figures look more comprehensive.

11. Evaluation plan and statistical safeguards

Separate data splits

Proposed seed namespaces: training `1001:1010`, validation `2001:2020`, and final test `3001:3050`. Record which namespace identifies the learned policy and which identifies the traffic/environment trace. Never use final test traces to choose parameters.

Use the same saved arrival traces for every policy. Separate traffic, controller exploration, contention choice, backoff, and channel randomness. Sharing a single seed is not enough because different policies consume different numbers of random draws. For stronger pairing, key environmental draws by request ID, attempt index, and event type, or save a replayable exogenous-draw table.

Test groups

1. Reference periodic-plus-event scenario.
2. Population sweep: 10k, 50k, 100k.
3. Event fraction sweep: 1%, 5%, 20%, 100%.
4. Held-out event widths and onsets, such as 1/5/20 s and different onsets from training.
5. Held-out synchronisation: uniform phases, clustered phases, near-simultaneous reporting.
6. Repeated events, with a separately defined longer horizon.
7. Delay/holding-time and observation-noise tests.
8. Coverage/repetition and later-stage bottleneck tests.
9. Action, reward, and state ablations.

Do not run the full Cartesian product initially. Start with the reference case and one-factor sweeps, then selected combined stress cases.

Fix the horizon/cohort issue explicitly

The methodology's `E(H)` includes matured deadlines plus early drops. At an arbitrary short horizon, early drops can make the denominator depend on policy, while early on-time completions with future deadlines are excluded.

For the **primary paired comparison**, generate a fixed arrival cohort, stop arrivals, and run until every cohort deadline has elapsed. With arrivals before 120 s and deadlines at most 60 s, H=180 s achieves this. Every policy is then scored on the same generated requests.

Also implement the methodology's exact interim metric when needed, but label it as an interim horizon metric. Report the cohort size, early drops, early completions, pending requests, and not-yet-matured deadlines. Never present policy-dependent interim denominators as an equivalent paired service comparison.

Report these outcomes

- Primary: on-time completion fraction for the fixed cohort, both overall and per traffic class.
- Its complement, miss fraction, labelled as the same information in another form—not a second independent gain.
- Total completions, late completions, drops by reason, unresolved requests, and backlog over time.
- Median, p95, and p99 delay among completed requests, always alongside service outcomes.
- Attempt-conditioned preamble success: successful preamble attempts / actual transmitted attempts.
- Collision measure: define whether it counts collided devices or collided choices. Keep radio failures, no response, and later-stage failures separate.
- Resource reservation/use and signalling changes.

- Table/policy memory, feature-extraction cost, decision latency, training samples, and wall-clock training time.

When a denominator is zero, return NaN/“not applicable,” not zero. A no-arrival scenario is not perfect service. Report energy only after implementing a physical power–time model; otherwise show attempts/repetitions with those names.

Use paired trace-level differences and confidence intervals. Account for both independent training-seed and test-trace variability—for example, with a documented hierarchical bootstrap. Do not treat thousands of requests from one correlated burst as thousands of independent training runs. Disclose invalid runs and their causes rather than silently selecting successful ones.

12. Tests that must pass before experiments

Test	Required result
One request, no competing traffic	Correct timestamps and completion at the represented endpoint.
No arrivals	No spurious completions; ratios undefined where appropriate.
All requests barred	All remain pending; no artificial success or disappearance.
Forced same-choice contention	Correct failed attempts and retained retry requests.
Uniform contention	Sample mean matches `M(1−1/n)^(M−1)` within Monte Carlo uncertainty under the same assumptions.
No-arrival drain	Under a no-drop, sufficient-capacity test, all pending requests eventually complete; under retry limits, every terminal loss is an explicit drop.
Backoff zero and maximum	Correct timer range; retry only at an eligible opportunity; no retroactive resampling.
Completion at deadline	`d−1 tick` and `d` on time; `d+1 tick` late.
Drop/miss reward accounting	Each event counted once; documented treatment of drop after an earlier miss.
Resource pressure	Infeasible/overlapping configurations rejected; no free unlimited Msg3/RAR service.
Delayed action	Old settings remain active until the logged effective time.

Test	Required result
Multiple messages per meter	FIFO retained; no duplicate simultaneous UE procedures.
Next-state replay	The logged Q-update state equals the next decision’s observation.
Terminal Q update	No continuation term after a true terminal transition.
Frozen policy	Q-table checksum unchanged during evaluation; epsilon is zero.
Repeated run	Same runtime, configuration, trace, and draw stream reproduce the result.
Runtime portability	MATLAB/Octave deterministic fixtures agree; stochastic equivalence tested without assuming identical generators.

Use a small event-log fixture whose expected records are written by hand. This catches errors that aggregate plots can hide.

13. Reviewer comments mapped to implementation deliverables

Reviewer concern	Planned resolution	Evidence required before claiming it is resolved
Different baselines in text, table, and figures	All outputs read `baseline_registry` and validated run IDs	Automatic check that each displayed series has a matching registered run and configuration.
DRL proxies	Remove synthetic/proxy result paths	Only actual trained algorithm logs can enter performance figures. Otherwise remove DRL superiority claims.
Collision and success repeat the same information	Define the exact events/denominators; prioritise request-level service and tail delay	Metric definitions and hand-checked fixtures.
Undefined `e`	Exclude ambiguous energy values	Either a documented physical model with units or an explicitly named attempt/repetition measure.

Reviewer concern	Planned resolution	Evidence required before claiming it is resolved
Incremental novelty	Test exact action, reward, and state contributions	Ablation results and comparison to closest work; no promise that implementation alone guarantees novelty.
Reward computation unclear	One documented `compute_reward` function, with event logs and tests	The same equation and terms appear in the methodology, pseudocode, diagram, and implementation.
Missing Algorithm 1	Maintain executable function flow and matching pseudocode	Code-to-method crosswalk with no missing steps.
Broken workflow arrows	Draw the ten-step event/learning sequence directly from the implemented flow	Manual verification, including the reward-to-update arrow and actual next-state transition.
Broken references, placeholder citation, spacing	Automated text checks plus author review	No unresolved figure/citation references or placeholder markers in the exported paper.
Cropped Figure 4	Generate vector plots with explicit page bounds	Inspect the actual exported two-column PDF at readable size, not just the plotting window.
Indicator-function objection	Keep valid notation and define 1/0 plainly	Correct request-based condition and boundary tests.
Poor/unverified AI-assisted presentation	Every numerical claim traces to logs; authors verify sources and figures	No hard-coded performance, fabricated uncertainty, invented citations, or unsupported claims.

This is a resolution plan, not a statement that the reviews have already been satisfied. Scientific contribution and presentation still require author judgement after the results exist.

14. Build order and acceptance gates

Phase 1 — Freeze the model and evidence map

Write the configuration sheet, source-to-parameter mapping, endpoint definition, cohort rule, and baseline registry. Archive the previous script unchanged. Resolve unknown protocol timings before Level B runs.

Gate: every important value has a source or a clearly labelled assumption; no undefined service endpoint.

Phase 2 — Build the request engine without learning

Implement arrivals, ledger, event order, backoff, per-meter FIFO, and Level A contention. Add conservation and boundary tests first.

Gate: all ledger tests and the analytical singleton check pass.

Phase 3 — Add Level B protocol/resource stages

Implement RAR, Msg3, contention resolution, timeouts, finite capacities, and effective-time scheduling. Validate against reference models where possible.

Gate: no hidden resource creation, impossible overlap, or premature completion; assumptions documented.

Phase 4 — Build fixed and adaptive baselines

Run B0 and B1 before learning. Establish capacity/drain curves and identify genuinely overloaded scenarios.

Gate: methods share the same environment and traces; baseline settings are not deliberately weak.

Phase 5 — Add tabular Q-learning

Implement observation encoding, feasible-action selection, reward events, actual next-state updates, checkpoints, and policy freezing. Run small pilots before scaling.

Gate: transition, reward, terminal-target, and frozen-policy tests pass; state/action coverage and runtime are logged.

Phase 6 — Tune and lock

Use training and validation only. Choose state bins, reward weights, learning budget, and baseline fixed controls under a declared tuning budget. Save a locked

experiment manifest.

Gate: final test traces have not influenced selection.

Phase 7 — Evaluate and perform ablations

Run independent training seeds and held-out paired traces. Add DQN only as a separately validated extension. Profile controller execution separately from simulation and disk logging.

Gate: uncertainty, failures, cohort counts, and per-class outcomes are available for every reported comparison.

Phase 8 — Generate the paper package

Build every figure and table from raw logs and the common registry. Produce the parameter table, Algorithm 1, workflow diagram, metric definitions, limitations, and reviewer-resolution matrix.

Gate: all plotted values trace to run IDs; final PDF inspected; no claim exceeds the implemented endpoint or validation evidence.

Do not promise an exact runtime or completion date before the Phase 2–3 profiling results. Use event queues, vectorised contention, preallocation, and checkpointed runs to control cost. Avoid scanning all historical request records at every millisecond.

15. What the first implementation should deliver

The first useful milestone is **not a high reward curve**. It is:

1. One reproducible periodic-plus-event trace.
2. A request ledger that passes conservation and deadline tests.
3. A validated contention model and explicitly defined service endpoint.
4. Fixed and adaptive baseline runs with honest backlog and request outcomes.
5. One trained and frozen tabular policy.
6. A comparison table generated from logs, with no placeholder or proxy results.

Only after that should the project expand to larger sweeps, richer states, coverage models, or a neural baseline.

Bottom line: start with portable `.m` code and tabular Q-learning, but make simulator correctness, fair comparison, and evidence-backed claims the core of the implementation. The earlier literature already contains joint and learning-based access control; this project must demonstrate the value and limits of its specific deadline-aware design rather than assume them.[7]